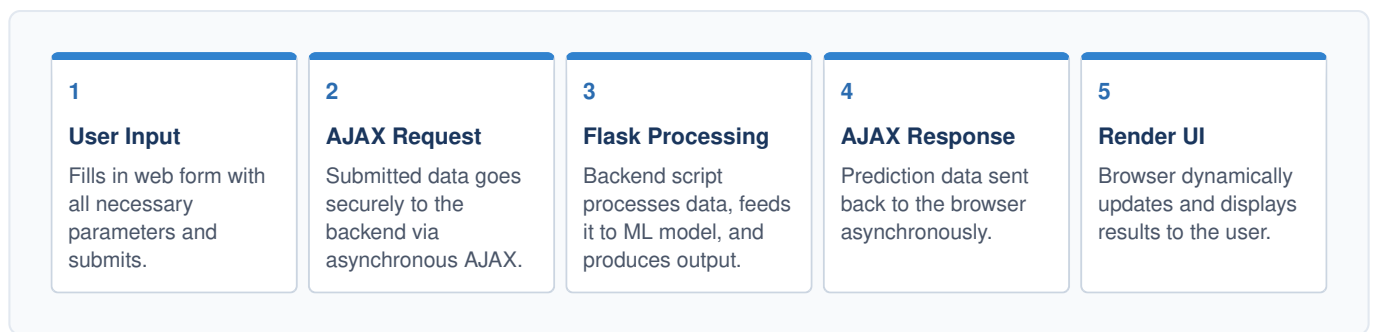


Technology Stack (Architecture & Stack)

Layer	Technology	Purpose
Frontend	HTML5, CSS3, Bootstrap, JavaScript	Provides a responsive and user-friendly interface for entering weather parameters and displaying flood prediction results.
Backend	Python, Flask	Handles user requests, processes input data, loads the trained machine learning model, and returns prediction results.
Machine Learning	Scikit-learn, XGBoost	Implements and trains classification models such as Decision Tree, Random Forest, K-Nearest Neighbours (KNN), and XGBoost for flood prediction.
Data Processing	Pandas, NumPy	Performs data cleaning, preprocessing, feature selection, and numerical computations.
Data Visualization	Matplotlib, Seaborn	Used during model development to analyze the dataset and visualize trends and model performance.
Model Storage	Pickle (.pkl)	Saves the trained machine learning model for deployment without retraining.
Dataset	Historical Weather and Flood Dataset (CSV)	Contains rainfall, cloud visibility, seasonal rainfall, and flood occurrence records used for training and testing.
Development Env	Jupyter Notebook, VS Code	Used for model training, experimentation, testing, and web application development.
Deployment Platform	IBM Cloud (Proposed)	Hosts the Flask web application, making it accessible online and scalable for real-world use.
Version Control	Git, GitHub	Manages source code, tracks changes, and supports collaborative development.

Interaction Model Overview



Architectural Description

The Flood Prediction System follows a layered architecture consisting of the frontend, backend, machine learning, and deployment layers. Users interact with the system through a web interface developed using HTML, CSS, Bootstrap, and JavaScript. The frontend sends weather parameters to the Flask backend using HTTP requests.

The backend preprocesses the input data using Pandas and NumPy before passing it to the trained machine learning model. Multiple classification algorithms—Decision Tree, Random Forest, K-Nearest Neighbours (KNN), and XGBoost—are evaluated during development, with the best-performing model (XGBoost) saved as a Pickle (.pkl) file and used for prediction. The prediction result is returned to the Flask application and displayed on the user interface, allowing authorities to quickly assess flood risk. The complete application is designed to be deployed on IBM Cloud, ensuring scalability, accessibility, and reliable operation for disaster management.